Comprehensive Project Report: Road Damage Detection & Smart Complaint Management System

This document contains the complete technical details, system architecture, and core codebase for the Road Damage Detection project.

File: `README.md`

```
# Road Damage Detection & Smart Complaint Management System

## Quick Start (Windows)
1. Double-click `run_backend.bat` to start the server.
2. Double-click `run_frontend.bat` to start the website.
3. Open http://localhost:5173 to use the app.

## Overview
This project is an AI-based system designed to detect road damage (like potholes, cracks) using a Deep Learning model (CNN). It automatically categorizes the damage severity and generates complaint tickets for authorities to review.

## Architecture
- **Frontend**: React (Vite) - User interface for uploading images and viewing reports.
- **Backend**: FastAPI (Python) - Handles API requests, processes images, and manages the database.
- **AI Model**: TensorFlow/MobilenetV2 - Pre-trained model fine-tuned for road damage detection.
- **Database**: SQLite - Stores ticket info and image references.

## Setup Instructions

### Prerequisites
- Python 3.8+
- Node.js & npm

### Backend Setup
1. `cd backend`
2. `py -m venv venv`
3. `venv\Scripts\activate`
4. `pip install -r requirements.txt`
5. `py -m uvicorn main:app --reload`

### Frontend Setup
1. `cd frontend`
2. `npm install`
3. `npm run dev`

### ML Model Training
```

1. `cd ml_model`
2. `py train.py`

File: `ARCHITECTURE.md`

```
# System Architecture

## Overview
The Road Damage Detection System uses a Microservices-like architecture where the Backend
(FastAPI) serves as the central hub connecting the Client (React), the AI Brain
(TensorFlow), and the Persistence Layer (SQLite).

## High-Level Architecture
```mermaid
graph TD
    Client[React Frontend] -->|HTTP/JSON| API[FastAPI Backend]
    API -->|Images| ML[ML Model (MobileNetV2)]
    ML -->|Predictions| API
    API -->|CRUD| DB[(SQLite Database)]
    API -->|Serve Static| Storage[File Storage]
```

Detailed Flow

1. Image Upload & Detection

1. User uploads image on React Frontend.
2. Image sent to `POST /upload/` on Backend.
3. Backend saves image to `static/uploads/`.
4. Backend calls `MLService`.
5. `MLService` preprocesses image (`224x224`, Normalized).
6. Model predicts Class (`Pothole`, `Crack`, `Normal`) and Confidence.
7. System calculates Severity based on Confidence & Type.
8. Ticket created in Database with status `Open`.
9. Response returned to Frontend.

2. Admin Management

1. Admin loads Dashboard.
2. Frontend requests `GET /tickets/`.
3. Backend queries Database.
4. List of tickets displayed in Table.
5. Admin clicks "Resolve".
6. Frontend sends `PUT /tickets/{id}` with `status="Resolved"`.
7. Backend updates Database.

Tech Stack

- **Frontend**: React, Vite, Recharts, Axios, CSS Modules.
- **Backend**: Python, FastAPI, SQLAlchemy, Pydantic.
- **AI/ML**: TensorFlow, Keras, MobileNetV2, OpenCV.
```

- **Database**: SQLite (File-based).

Directory Structure

```
/
├── backend/
│   ├── main.py          # API Routes
│   ├── models.py        # DB Tables
│   ├── schemas.py       # Pydantic Models
│   ├── ml_integration.py# Model Wrapper
│   └── static/          # Uploaded Images
├── frontend/
│   ├── src/
│   │   ├── pages/       # Dashboard & Upload
│   │   ├── App.jsx      # Wiring
│   │   └── main.jsx     # Entry
├── ml_model/
│   ├── train.py         # Training Script
│   ├── inference.py     # Prediction Logic
│   └── dataset/         # Training Data
```

File: `backend/main.py`

```python
from fastapi import FastAPI, Depends, HTTPException, UploadFile, File, Header, Form, Request
from fastapi.encoders import jsonable_encoder
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from sqlalchemy.orm import Session
from datetime import datetime, timedelta
from passlib.context import CryptContext
from jose import JWTError, jwt
import shutil
import os
import uuid

import database
import models
import schemas
import crud
from email_service import EmailService

email_service = EmailService()

# ---------------- DATABASE ----------------

models.Base.metadata.create_all(bind=database.engine)
app = FastAPI(title="Road Damage Detection API")
```

```
# Mount static folder
app.mount("/static", StaticFiles(directory="static"), name="static")

# ---------------- CORS ----------------

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ---------------- JWT SETTINGS ----------------

SECRET_KEY = "road_damage_secret_key"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60

pwd_context = CryptContext(schemes=["pbkdf2_sha256"], deprecated="auto")

# ---------------- UPLOAD FOLDER ----------------

UPLOAD_DIR = "static/uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

# ---------------- DATABASE DEPENDENCY ----------------

def get_db():
    db = database.SessionLocal()
    try:
        yield db
    finally:
        db.close()

# ---------------- PASSWORD FUNCTIONS ----------------

def hash_password(password: str):
    return pwd_context.hash(password)

def verify_password(plain, hashed):
    return pwd_context.verify(plain, hashed)

# ---------------- TOKEN FUNCTIONS ----------------

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(token: str, db: Session):
```

```
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        email = payload.get("sub")

        if email is None:
            raise HTTPException(status_code=401, detail="Invalid token")

    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")

    user = crud.get_user_by_email(db, email)

    if not user:
        raise HTTPException(status_code=401, detail="User not found")

    return user

# ---------------- ROOT ----------------

@app.get("/")
def root():
    return {"message": "Road Damage Detection API is running"}

# ---------------- REGISTER ----------------

@app.post("/register", response_model=schemas.UserResponse)
def register(user: schemas.UserCreate, db: Session = Depends(get_db)):

    if crud.get_user_by_email(db, user.email):
        raise HTTPException(status_code=400, detail="Email already registered")

    hashed_pw = hash_password(user.password)
    return crud.create_user(db, user.name, user.email, hashed_pw, user.phone)

# ---------------- USER PROFILE ----------------

@app.get("/me", response_model=schemas.UserResponse)
def get_my_profile(
    authorization: str = Header(None),
    db: Session = Depends(get_db)
):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Invalid token")
    token = authorization.split(" ")[1]
    user = get_current_user(token, db)
    return user

@app.put("/me/update", response_model=schemas.UserResponse)
def update_profile(
    user_update: schemas.UserUpdate,
    authorization: str = Header(None),
    db: Session = Depends(get_db)
```

```
):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Invalid token")
    token = authorization.split(" ")[1]
    user = get_current_user(token, db)

    updated_user = crud.update_user_profile(db, user.id, user_update.dict(exclude_unset=True))
    return updated_user

# ---------------- LOGIN ----------------

@app.post("/login")
def login(user: schemas.UserLogin, db: Session = Depends(get_db)):

    db_user = crud.get_user_by_email(db, user.email)

    if not db_user or not verify_password(user.password, db_user.password):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    access_token = create_access_token({"sub": db_user.email, "role": db_user.role})

    return {
        "access_token": access_token,
        "token_type": "bearer",
        "role": db_user.role,
        "name": db_user.name
    }

# ---------------- UPLOAD IMAGE (FIXED VERSION) ----------------

from typing import Optional

@app.post("/upload/")
async def upload_image(
    file: UploadFile = File(...),
    authorization: str = Header(None),
    db: Session = Depends(get_db)
):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Invalid token format")

    token = authorization.split(" ")[1]
    user = get_current_user(token, db)

    # Save uploaded file
    file_ext = file.filename.split(".")[-1]
    filename = f"{uuid.uuid4()}.{file_ext}"
    file_path = os.path.join(UPLOAD_DIR, filename)

    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)
```

```
    # Web-safe path for the frontend (database storage)
    db_image_path = f"/{UPLOAD_DIR}/{filename}"

    # Integrate real ML Model
    try:
        from ml_integration import MLService
        ml_service = MLService.get_instance()
        prediction = ml_service.predict(file_path)

        if "error" in prediction:
            if os.path.exists(file_path):
                os.remove(file_path)
            from fastapi.responses import JSONResponse
            return JSONResponse(status_code=400, content={"detail": prediction["error"]})

    except HTTPException:
        raise
    except Exception as e:
         if os.path.exists(file_path):
             os.remove(file_path)
         raise HTTPException(status_code=500, detail=f"ML Model Error: {str(e)}")

    return jsonable_encoder({
        "message": "Analysis ready",
        "prediction": prediction,
        "image_path": db_image_path
    })

# ---------------- CREATE TICKET (STEP 2) ----------------

@app.post("/tickets/", response_model=schemas.TicketResponse)
def create_ticket_endpoint(
    ticket_in: schemas.TicketCreate,
    authorization: str = Header(None),
    db: Session = Depends(get_db)
):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Invalid token format")
    token = authorization.split(" ")[1]
    user = get_current_user(token, db)

    prediction_data = {
        "class": ticket_in.damage_type,
        "severity": ticket_in.severity,
        "confidence": ticket_in.confidence,
        "location": ticket_in.location,
        "latitude": ticket_in.latitude,
        "longitude": ticket_in.longitude,
        "user_comment": ticket_in.user_comment
    }

    ticket = crud.create_ticket(db, user.id, prediction_data, ticket_in.image_path)
```

```
    # Send email notification
    try:
        email_service.send_confirmation(
            recipient_email=user.email,
            user_name=user.name,
            ticket_id=ticket.id,
            damage_type=ticket.damage_type,
            severity=ticket.severity
        )
    except Exception as e:
        print(f"Email Error: {e}")

    return ticket

# ---------------- GET MY TICKETS ----------------

@app.get("/my-tickets")
def get_my_tickets(
    authorization: str = Header(None),
    db: Session = Depends(get_db)
):

    if not authorization:
        raise HTTPException(status_code=401, detail="Authorization header missing")

    if not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Invalid token format")

    token = authorization.split(" ")[1]

    user = get_current_user(token, db)

    print(f"[DEBUG] Fetching tickets for user_id: {user.id} ({user.email})")
    tickets = crud.get_user_tickets(db, user.id)
    return jsonable_encoder(tickets)

# ---------------- ADMIN TICKETS ----------------
from pydantic import BaseModel
from typing import Optional

class TicketUpdate(BaseModel):
    status: Optional[str] = None
    admin_feedback: Optional[str] = None

@app.get("/tickets/")
def get_all_tickets(
    status: str = "All",
    severity: str = "All",
    is_escalated: Optional[int] = None,
    search: Optional[str] = None,
    db: Session = Depends(get_db)
```

```
):
    # Trigger escalation check on dashboard load
    crud.check_for_escalations(db)
    tickets = crud.get_all_tickets(db, status=status, severity=severity,
is_escalated=is_escalated, search=search)
    return jsonable_encoder(tickets)

@app.post("/tickets/{ticket_id}/verify_resolution/")
async def verify_resolution(
    ticket_id: int,
    file: UploadFile = File(...),
    authorization: str = Header(None),
    db: Session = Depends(get_db),
    request: Request = None
):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Invalid token format")

    token = authorization.split(" ")[1]

    # Ensure ticket exists
    ticket = db.query(models.Ticket).filter(models.Ticket.id == ticket_id).first()
    if not ticket:
        raise HTTPException(status_code=404, detail="Ticket not found")

    # Save uploaded file
    file_ext = file.filename.split(".")[-1]
    filename = f"resolve_{ticket_id}_{uuid.uuid4()}.{file_ext}"
    file_path = os.path.join(UPLOAD_DIR, filename)

    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    db_image_path = f"/{UPLOAD_DIR}/{filename}"

    # Verify with ML Service
    try:
        from ml_integration import MLService
        ml_service = MLService.get_instance()
        prediction = ml_service.predict(file_path)

        if "error" in prediction:
            if os.path.exists(file_path):
                os.remove(file_path)
            return {"success": False, "detail": prediction["error"]}

        detected_class = prediction.get("class", "")

        if detected_class == "Normal Road Surface":
            # AI PASSED: Auto-resolve ticket
            updated_ticket = crud.update_ticket(
                db=db,
```

```
                ticket_id=ticket_id,
                status="Resolved",
                resolution_image_path=db_image_path,
                verification_status="Passed"
            )

            # Send resolution confirmed email
            try:
                email_service.send_resolution_confirmed(
                    recipient_email=ticket.user.email,
                    user_name=ticket.user.name,
                    ticket_id=ticket.id,
                    damage_type=ticket.damage_type
                )
            except Exception as e:
                print(f"Email Error: {e}")

            return {
                "success": True,
                "message": "✅ AI Verification Passed! Road is fixed. Ticket auto-resolved.",
                "ticket": jsonable_encoder(updated_ticket)
            }
        else:
            # AI FAILED: Ask user to verify
            # Generate a secure one-time token
            verification_token = str(uuid.uuid4())

            updated_ticket = crud.update_ticket(
                db=db,
                ticket_id=ticket_id,
                status="In Progress",
                resolution_image_path=db_image_path,
                verification_status="AwaitingUserInput"
            )
            # Save token to ticket
            db_ticket = db.query(models.Ticket).filter(models.Ticket.id == ticket_id).first()
            db_ticket.user_verification_token = verification_token
            db.commit()
            db.refresh(db_ticket)

            # Build email verification links
            base_url = f"http://localhost:8000"
            after_image_url = f"{base_url}{db_image_path}"
            verify_yes_url = f"{base_url}/tickets/{ticket_id}/confirm_resolution?token=
{verification_token}&resolved=true"
            verify_no_url = f"{base_url}/tickets/{ticket_id}/confirm_resolution?token=
{verification_token}&resolved=false"

            try:
                email_service.send_user_verification_email(
                    recipient_email=ticket.user.email,
                    user_name=ticket.user.name,
```

```
                    ticket_id=ticket.id,
                    damage_type=ticket.damage_type,
                    after_image_url=after_image_url,
                    verify_yes_url=verify_yes_url,
                    verify_no_url=verify_no_url
                )
            except Exception as e:
                print(f"Email Error: {e}")

            return {
                "success": False,
                "message": f"⚠️ AI still detects: {detected_class}. Verification email sent to
user for manual confirmation.",
                "ticket": jsonable_encoder(db_ticket)
            }

    except Exception as e:
         if os.path.exists(file_path):
             os.remove(file_path)
         raise HTTPException(status_code=500, detail=f"Verification Error: {str(e)}")


@app.get("/tickets/{ticket_id}/confirm_resolution")
def confirm_resolution(
    ticket_id: int,
    token: str,
    resolved: bool,
    db: Session = Depends(get_db)
):
    """User clicks Yes/No from their verification email. Returns a styled HTML page."""
    from fastapi.responses import HTMLResponse

    ticket = db.query(models.Ticket).filter(models.Ticket.id == ticket_id).first()

    def error_page(msg):
        return HTMLResponse(content=f"""
        <html><body style="font-family:Arial,sans-serif;display:flex;align-
items:center;justify-content:center;min-height:100vh;background:#0f172a;color:#f8fafc;text-
align:center;">
        <div><div style="font-size:64px;margin-bottom:16px;">⚠️</div><h2>{msg}</h2></div>
        </body></html>
        """, status_code=400)

    if not ticket:
        return error_page("Ticket not found.")
    if ticket.user_verification_token != token:
        return error_page("Invalid or expired verification link.")
    if ticket.verification_status not in ("AwaitingUserInput",):
        # Already responded
        return HTMLResponse(content=f"""
        <html><body style="font-family:Arial,sans-serif;display:flex;align-
items:center;justify-content:center;min-height:100vh;background:#0f172a;color:#f8fafc;text-
```

```
align:center;">
        <div><div style="font-size:64px;">✅</div><h2>You've already responded to this ticket.
</h2><p style="color:#94a3b8;">Ticket #{ticket_id} status: {ticket.status}</p></div>
        </body></html>
        """)

    from datetime import datetime as dt
    ticket.user_verified_at = dt.utcnow()
    ticket.user_verification_token = None  # Invalidate token after use

    if resolved:
        ticket.status = "Resolved"
        ticket.verification_status = "UserConfirmedResolved"
        db.commit()

        try:
            email_service.send_status_update(
                recipient_email=ticket.user.email,
                user_name=ticket.user.name,
                ticket_id=ticket.id,
                new_status="Resolved",
                admin_feedback="You confirmed the road repair is complete. Thank you!"
            )
        except Exception as e:
            print(f"Email Error: {e}")

        return HTMLResponse(content=f"""
        <html><body style="font-family:Arial,sans-serif;display:flex;align-
items:center;justify-content:center;min-height:100vh;background:#0f172a;color:#f8fafc;text-
align:center;">
        <div style="max-width:500px;">
            <div style="font-size:72px;">✅</div>
            <h1 style="color:#4ade80;">Thank You!</h1>
            <p style="color:#94a3b8;font-size:1.1rem;">You confirmed that Ticket #{ticket_id}
is resolved.<br/>This ticket is now <strong style="color:#4ade80;">closed</strong>. Thank you
for making our roads safer!</p>
        </div>
        </body></html>
        """)
    else:
        ticket.status = "Escalated"
        ticket.verification_status = "UserEscalated"
        ticket.is_escalated = 1
        ticket.escalated_at = dt.utcnow()
        db.commit()

        # Notify user their escalation was registered
        try:
            email_service.send_user_escalation_confirmed(
                recipient_email=ticket.user.email,
                user_name=ticket.user.name,
                ticket_id=ticket.id
```

```
            )
        except Exception as e:
            print(f"Email Error (user): {e}")

        # Alert admin/authority
        try:
            email_service.send_escalation_alert(
                recipient_email=ticket.user.email,  # In production: admin email
                ticket_id=ticket.id,
                damage_type=ticket.damage_type,
                severity=ticket.severity,
                location=ticket.location,
                created_at=str(ticket.created_at)
            )
        except Exception as e:
            print(f"Email Error (admin): {e}")

        return HTMLResponse(content=f"""
        <html><body style="font-family:Arial,sans-serif;display:flex;align-
items:center;justify-content:center;min-height:100vh;background:#0f172a;color:#f8fafc;text-
align:center;">
        <div style="max-width:500px;">
            <div style="font-size:72px;">🚨</div>
            <h1 style="color:#f87171;">Issue Escalated</h1>
            <p style="color:#94a3b8;font-size:1.1rem;">We've escalated Ticket #{ticket_id} to
<strong style="color:#f87171;">higher authorities</strong>.<br/>You will be notified once
action is taken.</p>
        </div>
        </body></html>
        """)


@app.post("/tickets/{ticket_id}/mark_opened")
def mark_ticket_opened(
    ticket_id: int,
    authorization: str = Header(None),
    db: Session = Depends(get_db)
):
    """Called when admin opens a ticket modal. Auto-moves Open → In Progress."""
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Invalid token format")

    ticket = db.query(models.Ticket).filter(models.Ticket.id == ticket_id).first()
    if not ticket:
        raise HTTPException(status_code=404, detail="Ticket not found")

    if ticket.status != "Open":
        return {"message": "No change needed", "status": ticket.status}

    ticket.status = "In Progress"
    db.commit()
    db.refresh(ticket)
```

```
    try:
        email_service.send_status_update(
            recipient_email=ticket.user.email,
            user_name=ticket.user.name,
            ticket_id=ticket.id,
            new_status="In Progress",
            admin_feedback="An administrator has opened your ticket and is now reviewing your
report."
        )
    except Exception as e:
        print(f"Email Error: {e}")

    return {"message": "Ticket marked In Progress", "ticket": jsonable_encoder(ticket)}


from pydantic import BaseModel as _BaseModel

class UserVerifyRequest(_BaseModel):
    resolved: bool

@app.post("/tickets/{ticket_id}/user_verify_app")
def user_verify_app(
    ticket_id: int,
    body: UserVerifyRequest,
    authorization: str = Header(None),
    db: Session = Depends(get_db)
):
    """In-app shortcut: logged-in user confirms resolution directly from MyTickets page."""
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Invalid token")
    token = authorization.split(" ")[1]
    user = get_current_user(token, db)

    ticket = db.query(models.Ticket).filter(models.Ticket.id == ticket_id,
models.Ticket.user_id == user.id).first()
    if not ticket:
        raise HTTPException(status_code=404, detail="Ticket not found or not yours")
    if ticket.verification_status != "AwaitingUserInput":
        raise HTTPException(status_code=400, detail="Ticket is not awaiting user
verification")

    from datetime import datetime as dt
    ticket.user_verified_at = dt.utcnow()
    ticket.user_verification_token = None

    if body.resolved:
        ticket.status = "Resolved"
        ticket.verification_status = "UserConfirmedResolved"
        db.commit()
        try:
            email_service.send_status_update(
```

```
                recipient_email=user.email, user_name=user.name,
                ticket_id=ticket.id, new_status="Resolved",
                admin_feedback="You confirmed the road repair is complete. Thank you!"
            )
        except Exception as e:
            print(f"Email Error: {e}")
        return {"message": "Ticket resolved. Thank you for confirming!"}
    else:
        ticket.status = "Escalated"
        ticket.verification_status = "UserEscalated"
        ticket.is_escalated = 1
        ticket.escalated_at = dt.utcnow()
        db.commit()
        try:
            email_service.send_user_escalation_confirmed(
                recipient_email=user.email, user_name=user.name, ticket_id=ticket.id
            )
        except Exception as e:
            print(f"Email Error: {e}")
        return {"message": "Issue escalated to higher authority."}


@app.put("/tickets/{ticket_id}")
def update_ticket_endpoint(ticket_id: int, ticket_update: TicketUpdate, db: Session =
Depends(get_db)):
    updated_ticket = crud.update_ticket(db, ticket_id, ticket_update.status,
ticket_update.admin_feedback)
    if not updated_ticket:
        raise HTTPException(status_code=404, detail="Ticket not found")

    # Send email notification
    try:
        email_service.send_status_update(
            recipient_email=updated_ticket.user.email,
            user_name=updated_ticket.user.name,
            ticket_id=updated_ticket.id,
            new_status=updated_ticket.status,
            admin_feedback=updated_ticket.admin_feedback
        )
    except Exception as e:
        print(f"Email Error: {e}")

    return updated_ticket

@app.delete("/tickets/{ticket_id}")
def delete_ticket_endpoint(ticket_id: int, db: Session = Depends(get_db)):
    success = crud.delete_ticket(db, ticket_id)
    if not success:
        raise HTTPException(status_code=404, detail="Ticket not found")
    return {"message": "Ticket deleted successfully"}

# ---------------- ANALYTICS ----------------
```

```
@app.get("/analytics/")
def get_analytics(db: Session = Depends(get_db)):
    tickets = crud.get_all_tickets(db)

    total = len(tickets)

    damage_types = {}
    severity_dist = {}
    status_dist = {}

    # Very simple trend: count by date string YYYY-MM-DD
    trend_dict = {}

    for t in tickets:
        # Damage types
        damage_types[t.damage_type] = damage_types.get(t.damage_type, 0) + 1
        # Severity
        severity_dist[t.severity] = severity_dist.get(t.severity, 0) + 1
        # Status
        status_dist[t.status] = status_dist.get(t.status, 0) + 1

        # Date trend
        date_str = t.created_at.strftime("%Y-%m-%d")
        trend_dict[date_str] = trend_dict.get(date_str, 0) + 1

    trend_list = [{"date": k, "count": v} for k, v in sorted(trend_dict.items())]

    return jsonable_encoder({
        "total_tickets": total,
        "damage_type_distribution": damage_types,
        "severity_distribution": severity_dist,
        "status_distribution": status_dist,
        "trend": trend_list,
        "tickets": tickets
    })
```

File: `backend/models.py`

```
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, Text
from sqlalchemy.orm import relationship
from datetime import datetime
from database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
```

```
    phone = Column(String, nullable=True) # Optional phone number
    password = Column(String, nullable=False)
    role = Column(String, default="user")  # "user" or "admin"
    created_at = Column(DateTime, default=datetime.utcnow)

    tickets = relationship("Ticket", back_populates="user")


class Ticket(Base):
    __tablename__ = "tickets"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    image_path = Column(String)
    damage_type = Column(String)
    severity = Column(String)
    confidence = Column(Float)
    priority = Column(String)
    status = Column(String, default="Open")
    location = Column(String, default="Unknown")
    latitude = Column(Float, nullable=True)   # Added for GPS Map View
    longitude = Column(Float, nullable=True)  # Added for GPS Map View
    user_comment = Column(String, default=None)
    created_at = Column(DateTime, default=datetime.utcnow)
    admin_feedback = Column(String, default=None)
    resolution_image_path = Column(String, nullable=True)
    verification_status = Column(String, default="Pending")
    is_escalated = Column(Integer, default=0)
    escalated_at = Column(DateTime, nullable=True)
    user_verification_token = Column(String, nullable=True, unique=True)
    user_verified_at = Column(DateTime, nullable=True)

    user = relationship("User", back_populates="tickets")
```

File: `backend/schemas.py`

```
from pydantic import BaseModel
from datetime import datetime
from typing import Optional


# ---------------- USER SCHEMAS ----------------

class UserCreate(BaseModel):
    name: str
    email: str
    phone: Optional[str] = None
    password: str
```

```python
class UserLogin(BaseModel):
    email: str
    password: str


class UserResponse(BaseModel):
    id: int
    name: str
    email: str
    phone: Optional[str] = None

    class Config:
        from_attributes = True

class UserUpdate(BaseModel):
    name: Optional[str] = None
    phone: Optional[str] = None


class Token(BaseModel):
    access_token: str
    token_type: str


# ---------------- TICKET SCHEMAS ----------------

class TicketBase(BaseModel):
    location: Optional[str] = "Unknown"
    latitude: Optional[float] = None
    longitude: Optional[float] = None
    user_comment: Optional[str] = None


class TicketCreate(TicketBase):
    image_path: str
    damage_type: str
    severity: str
    confidence: float

class TicketResponse(TicketBase):
    id: int
    image_path: str
    damage_type: str
    severity: str
    confidence: float
    priority: str
    status: str
    created_at: datetime
    admin_feedback: Optional[str]
    user_comment: Optional[str]
    resolution_image_path: Optional[str] = None
    verification_status: Optional[str] = None
```

```
    is_escalated: int = 0
    escalated_at: Optional[datetime] = None

    # Adding user details for admin dashboard
    user: Optional[UserResponse] = None

    class Config:
        from_attributes = True
```

File: `backend/crud.py`

```
from sqlalchemy.orm import Session
import models
from datetime import datetime


def create_user(db: Session, name: str, email: str, hashed_password: str, phone: str =
None):
    user = models.User(name=name, email=email, password=hashed_password, phone=phone)
    db.add(user)
    db.commit()
    db.refresh(user)
    return user


def get_user_by_email(db: Session, email: str):
    return db.query(models.User).filter(models.User.email == email).first()

def update_user_profile(db: Session, user_id: int, user_update: dict):
    user = db.query(models.User).filter(models.User.id == user_id).first()
    if user:
        if "name" in user_update and user_update["name"]:
            user.name = user_update["name"]
        if "phone" in user_update and user_update["phone"] is not None:
             user.phone = user_update["phone"]
        db.commit()
        db.refresh(user)
    return user


def create_ticket(db: Session, user_id: int, ticket_data: dict, image_path: str):
    ticket = models.Ticket(
        user_id=user_id,
        image_path=image_path,
        damage_type=ticket_data["class"],
        severity=ticket_data["severity"],
        confidence=ticket_data["confidence"],
        priority=calculate_priority(ticket_data["class"], ticket_data["severity"]),
        location=ticket_data.get("location", "Unknown"),
        latitude=ticket_data.get("latitude", None),
```

```
        longitude=ticket_data.get("longitude", None),
        user_comment=ticket_data.get("user_comment", None)
    )
    db.add(ticket)
    db.commit()
    db.refresh(ticket)
    return ticket


def get_user_tickets(db: Session, user_id: int):
    from sqlalchemy.orm import joinedload
    return
db.query(models.Ticket).options(joinedload(models.Ticket.user)).filter(models.Ticket.user_id
== user_id).all()


def calculate_priority(damage_type, severity):
    if damage_type == "Normal":
        return "Low"
    if severity == "High":
        return "Critical"
    elif severity == "Medium":
        return "High"
    else:
        return "Medium"

def get_all_tickets(db: Session, status: str = None, severity: str = None, is_escalated: int
= None, search: str = None):
    from sqlalchemy.orm import joinedload
    query = db.query(models.Ticket).options(joinedload(models.Ticket.user))

    if status and status != "All":
        query = query.filter(models.Ticket.status == status)
    if severity and severity != "All":
        query = query.filter(models.Ticket.severity == severity)
    if is_escalated is not None:
        query = query.filter(models.Ticket.is_escalated == is_escalated)
    if search:
        query = query.filter(
            (models.Ticket.location.ilike(f"%{search}%")) |
            (models.Ticket.damage_type.ilike(f"%{search}%")) |
            (models.Ticket.id.cast(models.String).ilike(f"%{search}%"))
        )

    return query.order_by(models.Ticket.created_at.desc()).all()

def check_for_escalations(db: Session):
    """Marks 'Open' tickets older than 48 hours as escalated."""
    from datetime import timedelta
    cutoff = datetime.utcnow() - timedelta(hours=48)

    unaddressed_tickets = db.query(models.Ticket).filter(
```

```
        models.Ticket.status == "Open",
        models.Ticket.is_escalated == 0,
        models.Ticket.created_at <= cutoff
    ).all()

    escalated_count = 0
    for ticket in unaddressed_tickets:
        ticket.is_escalated = 1
        ticket.escalated_at = datetime.utcnow()
        escalated_count += 1

    if escalated_count > 0:
        db.commit()
    return escalated_count

def update_ticket(db: Session, ticket_id: int, status: str = None, admin_feedback: str =
None, resolution_image_path: str = None, verification_status: str = None):
    ticket = db.query(models.Ticket).filter(models.Ticket.id == ticket_id).first()
    if ticket:
        if status:
            ticket.status = status
        if admin_feedback is not None:
            ticket.admin_feedback = admin_feedback
        if resolution_image_path is not None:
            ticket.resolution_image_path = resolution_image_path
        if verification_status is not None:
            ticket.verification_status = verification_status
        db.commit()
        db.refresh(ticket)
    return ticket

def delete_ticket(db: Session, ticket_id: int):
    ticket = db.query(models.Ticket).filter(models.Ticket.id == ticket_id).first()
    if ticket:
        db.delete(ticket)
        db.commit()
        return True
    return False
```

File: `backend/database.py`

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

SQLALCHEMY_DATABASE_URL = "sqlite:///./road_damage.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
```

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()
```

File: `backend/email_service.py`

```
import smtplib
import ssl
import os
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from dotenv import load_dotenv

load_dotenv()

class EmailService:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(EmailService, cls).__new__(cls)
            cls._instance._init_service()
        return cls._instance

    def _init_service(self):
        self.smtp_server = os.getenv("SMTP_SERVER", "smtp.gmail.com")
        self.smtp_port = int(os.getenv("SMTP_PORT", 465))
        self.sender_email = os.getenv("SENDER_EMAIL")
        self.sender_password = os.getenv("SENDER_PASSWORD")
        self.enabled = all([self.sender_email, self.sender_password])

        if not self.enabled:
            print("[WARNING] Email Service disabled: SENDER_EMAIL or SENDER_PASSWORD not set
in .env")

    def _send_email(self, recipient_email, subject, body_html):
        if not self.enabled:
            print(f"[INTERNAL] Email suppressed (Service Disabled). Recipient:
{recipient_email}")
            return False

        message = MIMEMultipart("alternative")
        message["Subject"] = subject
        message["From"] = f"Road Damage Monitoring <{self.sender_email}>"
        message["To"] = recipient_email

        part = MIMEText(body_html, "html")
        message.attach(part)

        context = ssl.create_default_context()
```

```
        try:
            with smtplib.SMTP_SSL(self.smtp_server, self.smtp_port, context=context) as
server:
                server.login(self.sender_email, self.sender_password)
                server.sendmail(self.sender_email, recipient_email, message.as_string())
            print(f"[SUCCESS] Email sent to {recipient_email}")
            return True
        except Exception as e:
            print(f"[ERROR] Failed to send email: {str(e)}")
            return False

    def send_confirmation(self, recipient_email, user_name, ticket_id, damage_type,
severity):
        subject = f"Ticket #{ticket_id}: Road Damage Report Received"
        body = f"""
        <html>
            <body style="font-family: Arial, sans-serif; line-height: 1.6; color: #333;">
                <div style="max-width: 600px; margin: 0 auto; border: 1px solid #ddd;
border-radius: 8px; overflow: hidden;">
                    <div style="background: #4f46e5; color: white; padding: 20px; text-
align: center;">
                        <h1 style="margin: 0;">Report Received</h1>
                    </div>
                    <div style="padding: 30px;">
                        <p>Hello <strong>{user_name}</strong>,</p>
                        <p>Thank you for reporting road damage. Your report has been
successfully logged in our system.</p>
                        <div style="background: #f9fafb; padding: 20px; border-radius: 6px;
margin: 20px 0;">
                            <p style="margin: 0;"><strong>Ticket ID:</strong> #{ticket_id}
</p>
                            <p style="margin: 5px 0;"><strong>Damage Type:</strong>
{damage_type}</p>
                            <p style="margin: 5px 0;"><strong>Severity:</strong> {severity}
</p>
                        </div>
                        <p>Our team will review the report and take necessary action. You
will receive an update once the status changes.</p>
                        <p>Safe travels!</p>
                    </div>
                    <div style="background: #f3f4f6; color: #666; padding: 15px; text-align:
center; font-size: 12px;">
                        This is an automated message from the Road Damage Detection System.
                    </div>
                </div>
            </body>
        </html>
        """
        return self._send_email(recipient_email, subject, body)

    def send_status_update(self, recipient_email, user_name, ticket_id, new_status,
admin_feedback=None):
```

```
        subject = f"Update on Ticket #{ticket_id}: Status is now {new_status}"
        feedback_section = f'<div style="background: #fff7ed; padding: 15px; border-left:
4px solid #f97316; margin: 20px 0;"><strong>Admin Feedback:</strong><br/>{admin_feedback}
</div>' if admin_feedback else ""

        body = f"""
        <html>
            <body style="font-family: Arial, sans-serif; line-height: 1.6; color: #333;">
                <div style="max-width: 600px; margin: 0 auto; border: 1px solid #ddd;
border-radius: 8px; overflow: hidden;">
                    <div style="background: #6366f1; color: white; padding: 20px; text-
align: center;">
                        <h1 style="margin: 0;">Ticket Update</h1>
                    </div>
                    <div style="padding: 30px;">
                        <p>Hello <strong>{user_name}</strong>,</p>
                        <p>There has been an update on your road damage report.</p>
                        <p style="font-size: 18px;">New Status: <span style="color: #4f46e5;
font-weight: bold;">{new_status}</span></p>
                        {feedback_section}
                        <p>You can view more details in the "My Tickets" section of the
application.</p>
                    </div>
                    <div style="background: #f3f4f6; color: #666; padding: 15px; text-align:
center; font-size: 12px;">
                        This is an automated message from the Road Damage Detection System.
                    </div>
                </div>
            </body>
        </html>
        """
        return self._send_email(recipient_email, subject, body)
    def send_escalation_alert(self, recipient_email, ticket_id, damage_type, severity,
location, created_at):
        subject = f"URGENT: Escalated Ticket #{ticket_id} - Unaddressed for 48 Hours"
        body = f"""
        <html>
            <body style="font-family: Arial, sans-serif; line-height: 1.6; color: #333;">
                <div style="max-width: 600px; margin: 0 auto; border: 2px solid #ef4444;
border-radius: 8px; overflow: hidden;">
                    <div style="background: #ef4444; color: white; padding: 20px; text-
align: center;">
                        <h1 style="margin: 0;">Escalation Alert</h1>
                    </div>
                    <div style="padding: 30px;">
                        <p>This is an automated escalation alert for a road damage report
that has remained <strong>Open</strong> for more than 48 hours without administrative
action.</p>
                        <div style="background: #fee2e2; padding: 20px; border-radius: 6px;
margin: 20px 0; border-left: 4px solid #ef4444;">
                            <p style="margin: 0;"><strong>Ticket ID:</strong> #{ticket_id}
</p>
```

```
                            <p style="margin: 5px 0;"><strong>Report Date:</strong>
{created_at}</p>
                            <p style="margin: 5px 0;"><strong>Damage Type:</strong>
{damage_type}</p>
                            <p style="margin: 5px 0;"><strong>Severity:</strong> {severity}
</p>
                            <p style="margin: 5px 0;"><strong>Location:</strong> {location}
</p>
                        </div>
                        <p>Please review this ticket immediately in the Admin Dashboard.</p>
                    </div>
                    <div style="background: #f3f4f6; color: #666; padding: 15px; text-align:
center; font-size: 12px;">
                        Higher Authority Notification System | Road Damage Detection
                    </div>
                </div>
            </body>
        </html>
        """
        return self._send_email(recipient_email, subject, body)

    def send_resolution_confirmed(self, recipient_email, user_name, ticket_id, damage_type):
        """Sent when AI verification passes — road confirmed fixed."""
        subject = f"✅ Ticket #{ticket_id}: Road Repair Confirmed!"
        body = f"""
        <html>
            <body style="font-family: Arial, sans-serif; line-height: 1.6; color: #333;">
                <div style="max-width: 600px; margin: 0 auto; border: 1px solid #22c55e;
border-radius: 12px; overflow: hidden;">
                    <div style="background: linear-gradient(135deg, #16a34a, #15803d);
color: white; padding: 30px; text-align: center;">
                        <div style="font-size: 48px; margin-bottom: 10px;">✅</div>
                        <h1 style="margin: 0; font-size: 1.8rem;">Road Repair Confirmed!
</h1>
                    </div>
                    <div style="padding: 30px;">
                        <p>Hello <strong>{user_name}</strong>,</p>
                        <p>Great news! Our AI system has verified that the road damage you
reported has been <strong>successfully repaired</strong>.</p>
                        <div style="background: #f0fdf4; padding: 20px; border-radius: 8px;
border-left: 4px solid #22c55e; margin: 20px 0;">
                            <p style="margin: 0;"><strong>Ticket ID:</strong> #{ticket_id}
</p>
                            <p style="margin: 5px 0;"><strong>Issue:</strong> {damage_type}
</p>
                            <p style="margin: 5px 0;"><strong>Status:</strong> <span
style="color: #16a34a; font-weight: bold;">Resolved</span></p>
                        </div>
                        <p>Thank you for helping keep our roads safe. Your report made a
difference!</p>
                        <p>Safe travels! 🚗</p>
                    </div>
```

```
                    <div style="background: #f3f4f6; color: #666; padding: 15px; text-align:
center; font-size: 12px;">
                        Road Damage Detection System — Automated Resolution Notice
                    </div>
                </div>
            </body>
        </html>
        """
        return self._send_email(recipient_email, subject, body)

    def send_user_verification_email(self, recipient_email, user_name, ticket_id,
damage_type,
                                     after_image_url, verify_yes_url, verify_no_url):
        """Sent when AI fails — ask user if the road is actually fixed."""
        subject = f"❓ Ticket #{ticket_id}: Is your road issue resolved?"
        body = f"""
        <html>
            <body style="font-family: Arial, sans-serif; line-height: 1.6; color: #333;
margin: 0; padding: 0;">
                <div style="max-width: 620px; margin: 0 auto; border: 1px solid #ddd;
border-radius: 12px; overflow: hidden;">

                    <div style="background: linear-gradient(135deg, #4f46e5, #7c3aed);
color: white; padding: 30px; text-align: center;">
                        <div style="font-size: 48px; margin-bottom: 10px;">🔍</div>
                        <h1 style="margin: 0; font-size: 1.6rem;">Please Verify Road
Repair</h1>
                        <p style="margin: 8px 0 0 0; opacity: 0.85;">Ticket #{ticket_id} ·
{damage_type}</p>
                    </div>

                    <div style="padding: 30px;">
                        <p>Hello <strong>{user_name}</strong>,</p>
                        <p>Our automatic AI analysis was <strong>unable to fully
confirm</strong> whether the road damage you reported has been repaired. We need
<strong>your confirmation</strong>.</p>

                        <p style="font-weight: 600; margin-bottom: 8px;">📸 After-repair
image submitted by the field team:</p>
                        <div style="border-radius: 10px; overflow: hidden; border: 2px solid
#e5e7eb; margin-bottom: 24px;">
                            <img src="{after_image_url}" alt="After Repair" style="width:
100%; display: block; object-fit: cover; max-height: 300px;" />
                        </div>

                        <p style="font-size: 1.1rem; font-weight: 700; text-align: center;
margin-bottom: 20px; color: #1e293b;">
                            Is the road issue at your location now resolved?
                        </p>

                        <table style="width: 100%; border-collapse: separate; border-
spacing: 12px;">
```

```
                        <tr>
                            <td style="width: 50%; text-align: center;">
                                <a href="{verify_yes_url}"
                                   style="display: block; background: linear-
gradient(135deg, #16a34a, #15803d); color: white; text-decoration: none;
                                          padding: 16px 10px; border-radius: 10px; font-
size: 1.1rem; font-weight: 700;">
                                    ✅ Yes, It's Fixed!
                                </a>
                            </td>
                            <td style="width: 50%; text-align: center;">
                                <a href="{verify_no_url}"
                                   style="display: block; background: linear-
gradient(135deg, #dc2626, #b91c1c); color: white; text-decoration: none;
                                          padding: 16px 10px; border-radius: 10px; font-
size: 1.1rem; font-weight: 700;">
                                    ❌ No, Still Broken
                                </a>
                            </td>
                        </tr>
                    </table>

                    <p style="margin-top: 24px; font-size: 0.85rem; color: #6b7280;
text-align: center;">
                        If you click <em>"No, Still Broken"</em>, this issue will be
<strong>escalated to higher authorities</strong> for urgent attention.
                    </p>
                </div>

                <div style="background: #f3f4f6; color: #999; padding: 15px; text-align:
center; font-size: 12px;">
                    Road Damage Detection System — User Verification Request
                </div>
            </div>
        </body>
    </html>
    """
    return self._send_email(recipient_email, subject, body)

def send_user_escalation_confirmed(self, recipient_email, user_name, ticket_id):
    """Sent to user after they report the issue is still not fixed."""
    subject = f"🚨 Ticket #{ticket_id}: Issue Escalated to Higher Authority"
    body = f"""
    <html>
        <body style="font-family: Arial, sans-serif; line-height: 1.6; color: #333;">
            <div style="max-width: 600px; margin: 0 auto; border: 2px solid #ef4444;
border-radius: 12px; overflow: hidden;">
                <div style="background: linear-gradient(135deg, #dc2626, #b91c1c);
color: white; padding: 30px; text-align: center;">
                    <div style="font-size: 48px; margin-bottom: 10px;">🚨</div>
                    <h1 style="margin: 0; font-size: 1.6rem;">Issue Escalated</h1>
                </div>
```

```
                    <div style="padding: 30px;">
                        <p>Hello <strong>{user_name}</strong>,</p>
                        <p>We've received your feedback that Ticket #{ticket_id} is
<strong>still not resolved</strong>.</p>
                        <p>This issue has been <strong>escalated to higher
authorities</strong> for urgent action. You will receive a follow-up once the issue is
properly addressed.</p>
                        <p>Thank you for keeping our roads safe.</p>
                    </div>
                    <div style="background: #f3f4f6; color: #666; padding: 15px; text-align:
center; font-size: 12px;">
                        Road Damage Detection System
                    </div>
                </div>
            </body>
        </html>
        """
        return self._send_email(recipient_email, subject, body)
```

File: `backend/ml_integration.py`

```python
import sys
import os

# Add ml_model to path
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'ml_model')))

from inference import RoadDamagePredictor

class MLService:
    _instance = None
    _predictor = None

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            cls._instance = MLService()
            # Initialize predictor
            # Get the project root directory (parent of backend)
            backend_dir = os.path.dirname(os.path.abspath(__file__))
            project_root = os.path.dirname(backend_dir)
            model_path = os.path.join(project_root, 'ml_model', 'road_damage_model.h5')

            # Debug: print the path
            print(f"[DEBUG] Model path: {model_path}")
            print(f"[DEBUG] Model exists: {os.path.exists(model_path)}")

            cls._predictor = RoadDamagePredictor(model_path)
        return cls._instance
```

```
    def predict(self, image_path):
        if self._predictor:
            return self._predictor.predict(image_path)
        return {"error": "Model not initialized"}
```

File: `frontend/package.json`

```
{
  "name": "road-damage-frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "test": "jest --env=jsdom"
  },
  "dependencies": {
    "axios": "^1.4.0",
    "leaflet": "^1.9.4",
    "lucide-react": "^0.263.1",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-leaflet": "^4.2.1",
    "react-router-dom": "^6.14.1",
    "recharts": "^2.7.2"
  },
  "devDependencies": {
    "@babel/preset-env": "^7.24.0",
    "@babel/preset-react": "^7.24.0",
    "@testing-library/jest-dom": "^6.0.0",
    "@testing-library/react": "^14.0.0",
    "@vitejs/plugin-react": "^4.0.1",
    "babel-jest": "^29.0.0",
    "jest": "^29.0.0",
    "jest-environment-jsdom": "^29.0.0",
    "vite": "^4.4.5"
  }
}
```

File: `ml_model/train.py`

```
import os
import argparse
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
```

```
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from dataset_utils import create_data_generators, IMG_SIZE, BATCH_SIZE, VALIDATION_SPLIT
import numpy as np
from sklearn.utils.class_weight import compute_class_weight

DATA_DIR = 'dataset'
MODEL_SAVE_PATH = 'road_damage_model.h5'
NUM_CLASSES = 2  # Binary classification: Pothole (1) or Normal (0)

def build_model(num_classes, learning_rate=0.0001, fine_tune=False):
    """Builds the model using MobileNetV2 as a base."""
    base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(IMG_SIZE,
IMG_SIZE, 3))

    # Fine-tuning: unfreeze last 30 layers for better accuracy
    if fine_tune:
        base_model.trainable = True
        for layer in base_model.layers[:-30]:
            layer.trainable = False
        print("✓ Fine-tuning enabled - will train last 30 layers")
    else:
        # Initial training: freeze base model
        base_model.trainable = False
        print("✓ Transfer learning mode - base model frozen")

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dropout(0.3)(x)  # Increased from 0.2
    x = Dense(256, activation='relu')(x)  # Increased from 128
    x = Dropout(0.3)(x)  # Added extra dropout layer
    x = Dense(128, activation='relu')(x)  # Added extra dense layer
    predictions = Dense(num_classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=predictions)

    model.compile(optimizer=Adam(learning_rate=learning_rate),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return model

def train(args):
    print(f"Preparing data generators from {DATA_DIR} with {VALIDATION_SPLIT*100}%
validation split...")
    try:
        train_gen, val_gen = create_data_generators(DATA_DIR, args.batch_size)
    except Exception as e:
        print(f"Error creating data generators: {e}")
        return
```

```
    if train_gen.samples == 0:
        print("No training data found in 'dataset' folder.")
        print("\n⚠️  ATTENTION: Please add training images first!")
        return

    num_classes = train_gen.num_classes
    print(f"Building model with MobileNetV2 base for {num_classes} classes...")
    model = build_model(num_classes, args.learning_rate, fine_tune=args.fine_tune)
    model.summary()

    print(f"\n✓ Training data found:")
    print(f"  Training samples: {train_gen.samples}")
    print(f"  Validation samples: {val_gen.samples}")
    print(f"  Classes: {train_gen.class_indices}")

    # Compute class weights to handle imbalanced dataset (e.g. 1000 potholes vs 70 cracks)
    class_weights = compute_class_weight(
        'balanced',
        classes=np.unique(train_gen.classes),
        y=train_gen.classes
    )
    class_weight_dict = dict(enumerate(class_weights))
    print(f"  Class weights: {class_weight_dict}")

    print(f"\nStarting training for {args.epochs} epochs...")
    print(f"Batch size: {args.batch_size}, Learning rate: {args.learning_rate}")

    callbacks = [
        ModelCheckpoint(MODEL_SAVE_PATH, save_best_only=True, monitor='val_loss',
mode='min', verbose=1),
        EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True, verbose=1),
        ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3, min_lr=1e-6,
verbose=1)
    ]

    history = model.fit(
        train_gen,
        steps_per_epoch=max(1, train_gen.samples // args.batch_size),
        validation_data=val_gen,
        validation_steps=max(1, val_gen.samples // args.batch_size),
        epochs=args.epochs,
        callbacks=callbacks,
        class_weight=class_weight_dict
    )

    print("\n" + "="*60)
    print("✓ Training complete!")
    print("="*60)
    print(f"Model saved to: {MODEL_SAVE_PATH}")
    print(f"Final validation accuracy: {history.history['val_accuracy'][-1]:.2%}")
    print("="*60)
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Train Road Damage Detection Model")
    parser.add_argument("--epochs", type=int, default=50, help="Number of epochs (default:
50)")
    parser.add_argument("--batch_size", type=int, default=16, help="Batch size (default:
16)")
    parser.add_argument("--learning_rate", type=float, default=0.0001, help="Learning rate
(default: 0.0001)")
    parser.add_argument("--fine_tune", action="store_true", help="Enable fine-tuning of base
model")

    args = parser.parse_args()
    train(args)
```

File: `ml_model/inference.py`

```
import os
import numpy as np
import cv2
from PIL import Image
import tensorflow as tf
from tensorflow.keras.models import load_model

MODEL_PATH = 'road_damage_model.h5'
IMG_SIZE = 224

# The model is trained on 4 classes: 0=crack, 1=non_road, 2=normal, 3=pothole
CLASSES = ['Crack', 'Non_Road', 'Normal', 'Pothole']  # Must match training order

# Severity scoring based on confidence bands
SEVERITY_BANDS = {
    'Pothole': {
        (0.92, 1.01): ('High', 'Critical road failure. Immediate repair required.'),
        (0.78, 0.92): ('Medium', 'Significant pothole detected. Schedule repair soon.'),
        (0.60, 0.78): ('Low', 'Minor pothole or surface distress. Monitor regularly.'),
    },
    'Crack': {
        (0.85, 1.01): ('Medium', 'Significant cracking. May lead to structural failure.'),
        (0.50, 0.85): ('Low', 'Minor surface cracking. Monitor regularly.'),
    },
    'Normal': {
        (0.0, 1.01): ('None', 'Road surface appears undamaged.'),
    }
}

def preprocess_image(image_path):
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError("Could not open image.")
```

```
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # 1. Resize
    img_resized = cv2.resize(img, (IMG_SIZE, IMG_SIZE))

    # 2. Denoising
    img_denoised = cv2.fastNlMeansDenoisingColored(img_resized, None, 10, 10, 7, 21)

    # 3. CLAHE (Contrast Limited Adaptive Histogram Equalization)
    lab = cv2.cvtColor(img_denoised, cv2.COLOR_RGB2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    cl = clahe.apply(l)
    limg = cv2.merge((cl,a,b))
    img_clahe = cv2.cvtColor(limg, cv2.COLOR_LAB2RGB)

    # 4. Normalize to [0,1]
    img_normalized = img_clahe / 255.0

    # Expand dims for batch size of 1
    return np.expand_dims(img_normalized, axis=0), img_resized

def is_road_like_image(img_array):
    if img_array.dtype != np.uint8:
        img_uint8 = (img_array * 255).astype(np.uint8)
    else:
        img_uint8 = img_array

    brightness = np.mean(img_uint8)
    if brightness < 30:
        return False, "Image is too dark. Please upload a well-lit road photo."
    if brightness > 220:
        return False, "Image is too bright or appears to be a document. Please upload a real
road photo."

    hsv = cv2.cvtColor(img_uint8, cv2.COLOR_RGB2HSV)
    mean_saturation = np.mean(hsv[:, :, 1])

    if mean_saturation > 60:
        return False, "This appears to be a face, indoor scene, or graphic (too much color).
Please upload a standard road photo."

    return True, "OK"

class RoadDamagePredictor:
    def __init__(self, model_path=MODEL_PATH):
        self.model_path = model_path
        self.model = None
        self._load_model()

    def _load_model(self):
        if os.path.exists(self.model_path):
```

```
            try:
                self.model = load_model(self.model_path)
                print("Model loaded successfully!")
            except Exception as e:
                print(f"Error loading model: {e}")
        else:
            print(f"Model not found at {os.path.abspath(self.model_path)}. Please run
train.py first.")

    def _get_severity(self, damage_class, confidence):
        bands = SEVERITY_BANDS.get(damage_class, {})
        for (low, high), (severity, description) in bands.items():
            if low <= confidence < high:
                return severity, description
        return "Low", "Minor damage detected."

    def _get_damage_type_label(self, base_class, confidence):
        if base_class == "Normal":
            return "Normal Road Surface"
        elif base_class == "Crack":
            return "Surface Cracking"
        elif base_class == "Pothole":
            if confidence >= 0.88:
                return "Pothole (Severe)"
            elif confidence >= 0.70:
                return "Pothole (Moderate)"
            else:
                return "Pothole (Minor)"
        return base_class

    def predict(self, image_path):
        if self.model is None:
            return {"error": "Model not loaded. Please retrain."}

        try:
            processed_img, raw_img = preprocess_image(image_path)
        except Exception as e:
            return {"error": f"Could not process image: {e}"}

        # Validate if it looks like a road
        is_road, reason = is_road_like_image(raw_img)
        if not is_road:
            return {"error": f"⚠️ Not a road image: {reason}"}

        try:
            predictions = self.model.predict(processed_img)

            predicted_class_idx = int(np.argmax(predictions[0]))
            confidence = float(predictions[0][predicted_class_idx])
            base_class = CLASSES[predicted_class_idx]

            # Reject non-road classifications
```

```
            if base_class == "Non_Road":
                return {"error": "⚠️ AI determined this is not a road surface."}

            if base_class == "Normal" and confidence < 0.60:
                 return {
                    "class": "Unknown",
                    "confidence": round(confidence, 4),
                    "severity": "Unknown",
                    "description": "Image is ambiguous or unclear. Could not confidently
classify.",
                    "damage_score": 0
                 }

            severity, description = self._get_severity(base_class, confidence)
            damage_label = self._get_damage_type_label(base_class, confidence)

            # Simple 0-10 severity score (10 = absolute worst)
            damage_score = 0
            if base_class != "Normal":
                damage_score = round(confidence * 10, 1)

            result = {
                "class": damage_label,
                "confidence": round(confidence, 4),
                "severity": severity,
                "description": description,
                "damage_score": damage_score
            }

            return result

        except Exception as e:
            return {"error": str(e)}

if __name__ == "__main__":
    import sys
    if len(sys.argv) > 1:
        img_path = sys.argv[1]
        predictor = RoadDamagePredictor()
        result = predictor.predict(img_path)
        print("\n===== PREDICTION RESULT =====")
        for k, v in result.items():
            print(f"  {k}: {v}")
        print("="*29)
    else:
        print("Usage: python inference.py <image_path>")
```

File: `ml_model/setup_yolo_dataset.py`

```python
import os
import urllib.request
import zipfile
import yaml

# Constants
DATASET_URL = "https://github.com/ultralytics/yolov5/releases/download/v1.0/coco128.zip" # Using COCO128 structure for demo, but we will configure it for Potholes
DATA_DIR = "dataset_yolo"

def setup_dataset():
    print(f"Setting up custom road damage YOLO dataset in {DATA_DIR}...")

    # Create directories
    os.makedirs(os.path.join(DATA_DIR, "images/train"), exist_ok=True)
    os.makedirs(os.path.join(DATA_DIR, "images/val"), exist_ok=True)
    os.makedirs(os.path.join(DATA_DIR, "labels/train"), exist_ok=True)
    os.makedirs(os.path.join(DATA_DIR, "labels/val"), exist_ok=True)

    # Note: In a real-world scenario, you would download the 10GB RDD2022 dataset
    # from roboflow or kaggle. For this codebase, we will set up the structure
    # and provide the YAML so it can be trained locally with user-provided images.

    # Create dataset.yaml
    yaml_content = {
        'path': os.path.abspath(DATA_DIR),  # absolute path
        'train': 'images/train',
        'val': 'images/val',
        'test': '',  # optional

        # Classes
        'names': {
            0: 'Crack',
            1: 'Pothole'
        }
    }

    yaml_path = "dataset.yaml"
    with open(yaml_path, 'w') as f:
        yaml.dump(yaml_content, f, default_flow_style=False)

    print(f"✓ Created {yaml_path}")
    print("\n⚠️  ATTENTION: To train a real model, you must place:")
    print(f"1. .jpg images into {os.path.join(DATA_DIR, 'images/train')}")
    print(f"2. YOLO .txt label files into {os.path.join(DATA_DIR, 'labels/train')}")
    print("Then run 'python train_yolo.py'")

if __name__ == "__main__":
    setup_dataset()
```

File: `ml_model/dataset.yaml`

```yaml
names:
  0: Crack
  1: Pothole
path: A:\road damage\ml_model\dataset_yolo
test: ''
train: images/train
val: images/val
```